

1. Project Overview

Express-Kart is a multi-vendor e-commerce platform built with Flask and SQLite. It provides a complete ecosystem for buyers, sellers, and platform administrators.

2. Project Folder Structure

```
f:\FlaskApp\Ecommers
■ app.py                # Main application entry point
■ config.py             # Application configuration (DB, Email, Razorpay, Admin)
■ auth_utils.py         # Authentication helper functions (OTP sending)
■ forgot_password.py    # Blueprint for password reset logic
■ init_db.py            # Database initialization script
■ ecommerce.db          # SQLite database file
■ requirements.txt       # Project dependencies
■■■■projects_document/  # Project documentation and guides
■   ■■■■README.md       # Quick start guide
■   ■■■■DOCUMENTATION.md # Comprehensive technical documentation
■   ■■■■deployment_guide.md # PythonAnywhere deployment steps
■   ■■■■Ecommers_Project_Documentation.pdf # Exported PDF
■
■■■■static/            # Static assets
■   ■■■■css/            # Stylesheets (style.css, auth.css, super_admin.css)
■   ■■■■uploads/        # User/Merchant uploaded images
■   ■■■■script.js       # Frontend interactive scripts
■
■■■■templates/         # Jinja2 HTML templates
■   ■■■■auth/           # Authentication templates
■   ■■■■merchant/       # Merchant panel templates
■   ■■■■super_admin/    # Super Admin panel templates
■   ■■■■user/           # User panel templates
■   ■■■■base.html       # Main layout template
■   ■■■■navbar.html     # Navbar component
■
■■■■utils/             # Backend utility scripts
■   ■■■■pdf_generator.py # Invoice/Document PDF generation logic
```

3. System Architecture

The application follows a modular Flask pattern with the following components: - **Core Engine (app.py)**: Handles routing, session management, and core business logic. - **Database (ecommerce.db)**: Relational SQLite database. - **Frontend**: Jinja2 templates for dynamic HTML rendering, Vanilla CSS for styling. - **Authentication**: Bcrypt hashing and OTP-based verification for accounts.

3. Database Schema

- `users`: Stores customer information and hashed passwords.
- `admin`: Stores merchant and super admin accounts with status (active/pending/blocked).
- `products`: Product listings linked to merchants.
- `orders & order_items`: Transaction tracking.
- `cart`: Temporary storage for user shopping sessions.
- `wishlist`: User-saved products.

4. User Roles & Features

4.1. Customers (Users)

- Account registration with OTP.
- Product search and category filtering.
- Persistent shopping cart and wishlist.
- Secure checkout via Razorpay.
- Order history and invoice (PDF) downloads.

4.2. Merchants (Sellers)

- Self-registration (requires Super Admin approval).
- Dashboard with sales analytics and stock monitoring.
- Product CRUD operations (Create, Read, Update, Delete).
- Real-time order notifications.
- Profit margin tracking (default 20%).

4.3. Super Admin

- Full control over platform entities.
- Merchant approval/blocking system.
- Global product inventory management.
- Secure, hidden access route: /admin-secure-access-xk9.

5. Security Features

- **Hidden Admin Route:** The Super Admin login is not linked in UI to prevent brute force.
- **Brute Force Protection:** IP-based lockout after 5 failed attempts on the admin route.
- **Password Safety:** No passwords are stored in plain text.
- **OTP Verification:** Ensures valid email addresses for all registrations.

6. Development & Customization

Managing Super Admin Credentials

Super Admin credentials are set in `config.py`:

```
SUPER_ADMIN_EMAIL = "admin@123"
SUPER_ADMIN_PASSWORD = "admin123"
```

Update these values to change the administrative login.

Customizing Styles

The project uses two main stylesheets: - `static/style.css`: Main UI and responsive layout. - `static/auth.css`: Authentication specific styling.

7. Deployment Configuration

Refer to `deployment_guide.md` for detailed instructions on hosting the platform on PythonAnywhere.